Rotaract Club of Xiamen University Malaysia

Rotary Club of Kuala Lumpur Dynamics

Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	UTeMku Cemerlang
Project / Solution Name	PaySah
Name (Leader)	Chong Kai Li
Email & Phone (Leader)	kailichong148@gmail.com | 0146312883
University Affiliation(s)	Universiti Teknikal Malaysia Melaka
Live Prototype URL	https://paysah.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/Moyu0717/PaySah

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

PaySah is a React 19 staff web app backed by a Node.js 24 / Express 5 API and PostgreSQL 17. Receipt images are validated and preprocessed, then stored in private Replit App Storage with SHA-256 integrity hashes. The receipt worker sends images to a RunPod Serverless vLLM endpoint running Qwen2.5-VL-7B-Instruct for structured field extraction; Tesseract.js CPU OCR independently cross-checks the result and acts as a fallback. Deterministic rules compare extracted fields with the order and team-created synthetic bank transactions. Cashier/Owner review controls payment status, a separate release action, and the activity log. Replit hosts the public app, API, worker, PostgreSQL database and private image storage. Customers can also send receipts through a Telegram bot or a prefilled Google Form; both forward the image to the same intake API on Replit, which stores it and starts automatic verification. The Orbyte interface is simulated; no live bank, payment gateway or Orbyte integration is claimed.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
TypeScript 5.9.3	Main language for web app, API, worker, scripts and tests	Apache-2.0
Node.js 24 (Replit 24.13.0; >=24 <25)	API and receipt worker runtime	MIT
SQL / PostgreSQL 17	Schema, migrations, seed data; no ORM	PostgreSQL License
Google Apps Script (V8)	Google Form forwarding script	Google Apps Script terms; script code Apache-2.0

Name & Version	Purpose in Project	License
HTML / CSS	Web UI markup and styles; no CSS framework	—
React 19.2.0 / React DOM 19.2.0	Cashier/Owner web interface	MIT
Express 5.1.0	REST API server	MIT
Zod 4.1.12	Request/response validation	MIT
pg 8.16.3	PostgreSQL client, transactions and queries	MIT
Multer 2.4.0	Receipt image upload handling	MIT
Sharp 0.35.5	Image validation, preprocessing and synthetic receipt rendering	Apache-2.0
Tesseract.js 7.0.0 / tesseract.js-core 7.0.0	CPU OCR fallback and cross-check	Apache-2.0
@tesseract.js-data/eng 1.0.0	English OCR language data	MIT (package); Apache-2.0 (Tesseract traineddata)
@replit/object-storage 1.0.0	Private Replit receipt-image storage	MIT
Vite 7.3.7	Frontend development and production build	MIT
tsx 4.20.6	Run TypeScript scripts and tests	MIT
Node.js node:test	Integration test runner (128 tests)	Part of Node.js; MIT
@playwright/test 1.64.1 / Playwright 1.64.1	Browser testing, desktop and mobile viewport	Apache-2.0
embedded-postgres 17.10.0-beta.17	Local PostgreSQL for development/testing	MIT wrapper; PostgreSQL License binaries
concurrently 9.2.5	Run API and web dev servers together	MIT
@types/node 24.10.0, @types/express 5.0.3, @types/multer 2.3.0, @types/pg 8.15.6, @types/react 19.2.2, @types/react-dom 19.2.2	TypeScript type definitions	MIT
Three.js 0.186.1 (+ @types/three 0.186.0)	3D story page in the web interface	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
RunPod Serverless API (/run; OpenAI-compatible vLLM endpoint)	Sends receipt image to GPU vision model and receives structured fields	Sponsor-provided credits
Telegram Bot API (setWebhook, getFile, sendMessage)	Customer sends the receipt photo to the PaySah bot; the server receives it by webhook, downloads the image and replies with a fixed English acknowledgement	Free
Google Forms + Apps Script services (FormApp, DriveApp, UrlFetchApp, LockService, PropertiesService, ScriptApp triggers)	Customer uploads the receipt through a prefilled Google Form; an on-submit trigger forwards the image to the PaySah intake API	Free
Replit App Storage API via @replit/object-storage	Private storage of original receipt images	Sponsor-provided Replit credits
No bank/payment gateway/commercial LLM API	No direct live bank or payment gateway connection; synthetic bank transactions are used	Not applicable

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Replit Deployments — Reserved VM	Hosts web app, API and receipt worker at https://paysah.replit.app	Reserved VM; shared 0.5 vCPU / 2 GiB RAM; region not specified
Replit Database (PostgreSQL)	Production orders, receipts, verification results and activity log	Included with deployment; separate from development DB
Replit App Storage	Private receipt-image bucket	Included with Replit; private bucket
RunPod Serverless (vLLM worker)	Runs Qwen2.5-VL-7B-Instruct for receipt extraction	24 GB GPU tier, max workers 1, pay-per-use; region not specified
Google Workspace (Forms, Drive, Apps Script)	Receipt intake form, uploaded-file storage and forwarding script	Free personal Google account; timezone Asia/Kuala_Lumpur

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Qwen2.5-VL-7B-Instruct (Qwen/Qwen2.5-VL-7B-Instruct; served by RunPod vLLM v2.29.0)	Alibaba Qwen team via Hugging Face; hosted on RunPod	Pre-trained, used as-is with fixed extraction prompt; not fine-tuned	Apache-2.0 per model card
Tesseract LSTM English model (eng.traineddata), via Tesseract.js 7.0.0	Tesseract OCR project / naptha tessdata package	Pre-trained CPU OCR fallback and cross-check	Tesseract.js Apache-2.0; eng.traineddata Apache-2.0 (npm package MIT)

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
PaySah synthetic receipt diagnostic set (synthetic-layouts.v2; 36 images)	Team scripts: scripts/generate-ocr-dataset.ts; evaluation/ocr-expanded/	Synthetic	Apache-2.0 project licence	No
PaySah demo fixtures: synthetic receipt images, orders, merchant accounts, bank feed and demo intake receipts	Team scripts; fixtures/ and db/seeds/	Synthetic	Apache-2.0 project licence	No

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
OpenAI Codex (GPT-6)	Architecture, application code, API, database, OCR worker, UI, tests, scripts, Telegram/Form intake back end and documentation	Major
Anthropic Claude Code (Claude Opus 5.5)	Code review, RunPod integration, GPU/CPU cross-check, deployment guidance, Google Apps Script and submission drafts	Moderate
Replit Agent / Assistant	Minor help during Replit project setup	Minor

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

Every change was run, not just read. The final build passed 128 integration tests with 0 skipped and 11 browser scenarios, and a clean reinstall passed a CPU deployment smoke test. Telegram and Google Form intake were also tested live end-to-end. Tests run against real PostgreSQL in isolated schemas and cover tenant isolation, duplicate evidence and order versioning. Claude independently reviewed Codex's code; findings were fixed and retested. The GPU model was measured on 36 synthetic receipts, and its tendency to guess unclear fields led us to add a CPU cross-check. Failed runs are kept, not deleted.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Credits + hosting platform (US$20 credit)	Public deployment on a Reserved VM with Replit PostgreSQL and private App Storage for receipt images; also receives Telegram webhooks and Google Form submissions.
RunPod	Credits + Serverless GPU platform	Serverless vLLM endpoint (24 GB GPU tier) running Qwen2.5-VL-7B-Instruct to read receipt fields; evaluated on 36 synthetic receipts and called by the live app, with automatic fallback to CPU OCR

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Replit is essential for the submission: it hosts the only public deployment, database and private image storage, and receives Telegram and Google Form submissions. Technically it is replaceable by any Node.js host with PostgreSQL and object storage, since the code has no Replit-only logic besides a storage adapter. RunPod is replaceable but improves accuracy: on 36 synthetic receipts the GPU model read 248/252 fields versus 190/252 for CPU OCR. If RunPod is slow or unavailable, the worker automatically falls back to CPU OCR, so the system keeps working, with more receipts sent to manual review.

Sponsor usage evidence: Replit live check 5/5 PASS (23 October 2026); RunPod endpoint metrics screenshot: 110 requests, 100 completed, 10 failed, 0 retried. Screenshots are provided in Appendix A.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype records order references, amounts, customer display names, receipt images, extracted receipt fields and staff actions. The test/demo data are synthetic and do not contain real customer or bank records. Receipt images are stored in private Replit App Storage and related records in Replit PostgreSQL; access is restricted to signed-in staff within the same shop. Telegram receipt images pass through Telegram, and Google Form uploads are stored in the form owner's Google Drive. Images are sent to the team's RunPod endpoint for extraction. We use tenant-scoped queries, Cashier/Owner role limits, hashed session tokens, secrets stored in Replit Secrets, SHA-256 image integrity and no public image URLs. For Telegram we store a hashed sender ID, not phone numbers. Prototype data are intended to be kept for judging only; a production retention period, consent/notice and deletion process remain to be defined.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

The vision-language model may guess unreadable values, while OCR can fail on blurred, rotated, cropped or low-resolution receipts. PaySah cross-checks GPU extraction with CPU OCR and routes disagreements or unreadable fields to the Owner. Deterministic rules and synthetic bank-transaction matching, rather than the AI model alone, determine whether evidence qualifies; duplicate images, wrong accounts, pending/scheduled transfers, missing bank records, partial payments and high-value cases require review. RunPod availability and cost are external dependencies, mitigated by CPU OCR fallback. Evaluation so far uses 36 synthetic receipts, so results do not establish real-world fraud-detection accuracy. There is no live bank API or live Orbyte integration. During live tests, RunPod cold starts sometimes exceeded the timeout, so the CPU fallback handled those receipts.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

Signature: 

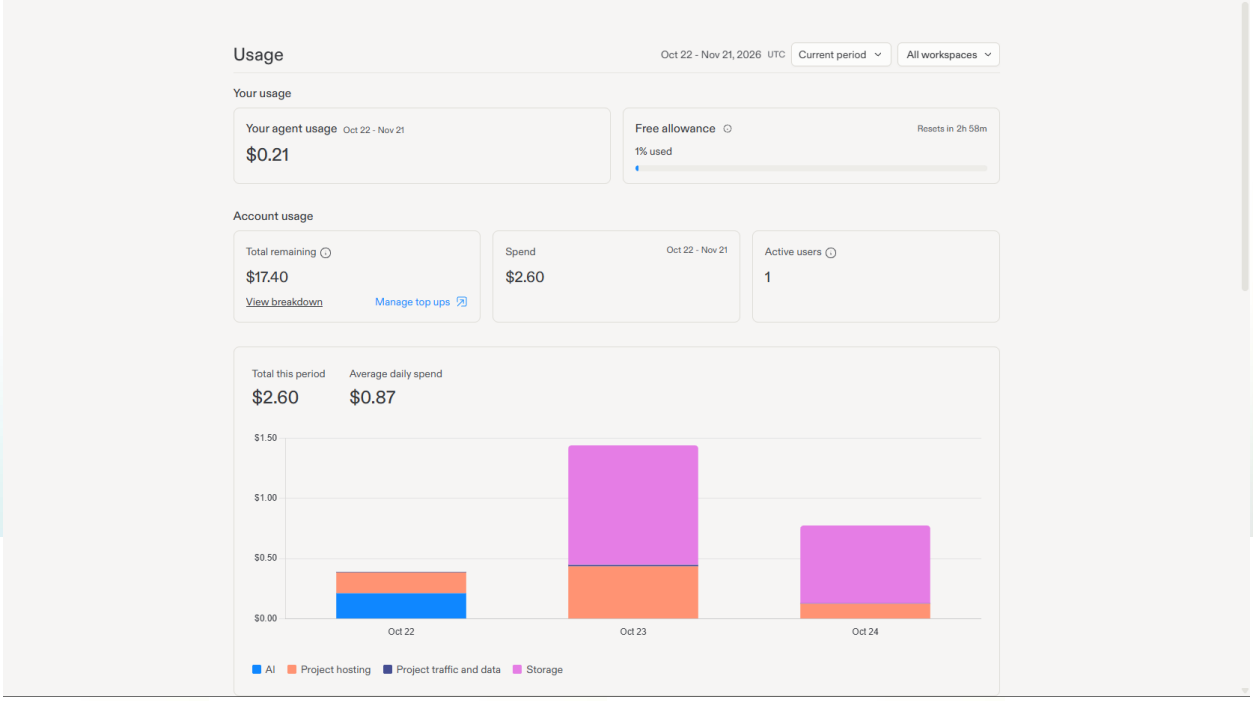
24 October 2026

Appendix A — Cloud Deployment and Usage Evidence

The following screenshots document the cloud usage and service status visible during the checks on 24 October 2026. Values are point-in-time observations and may change.

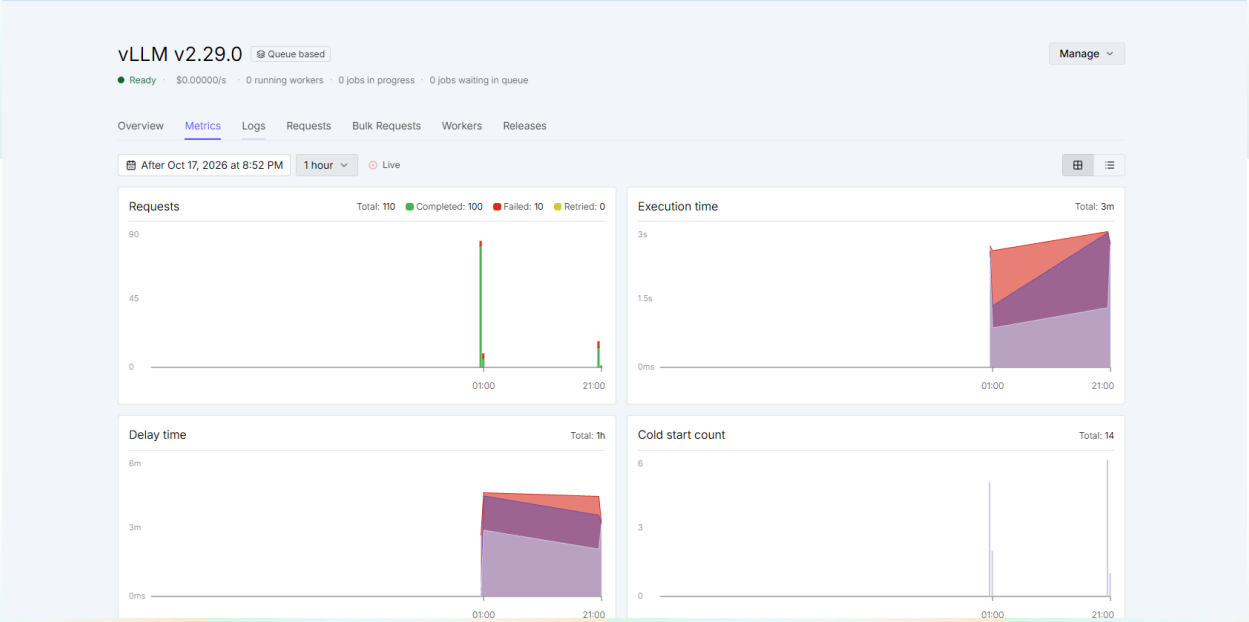
A1. Replit Usage

The dashboard shows $2.60 account spend and $17.40 remaining for the displayed period. It separately reports $0.21 in Replit Agent usage; these are distinct dashboard figures captured at that time.



A2. RunPod vLLM Endpoint Metrics

Screenshot shows 110 requests in the selected metrics view: 100 completed, 10 failed, and 0 retried. It also shows 14 cold starts, a current endpoint rate of $0.00000/s, 0 running workers, 0 jobs in progress, and 0 jobs waiting in queue at the time of capture.



A3. RunPod Account Balance and Spend Rate

Screenshot shows an account balance of $24.31. The endpoint metrics screenshot separately shows a $0.00000/s current rate, with 0 running workers, at the time of capture. The displayed account spend rate is $1.100/hr; all values are point-in-time snapshots.

